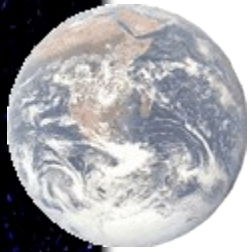
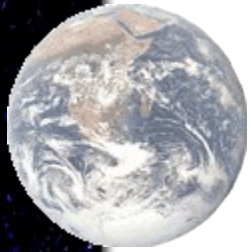


Concurrency: Mutual Exclusion and Synchronization



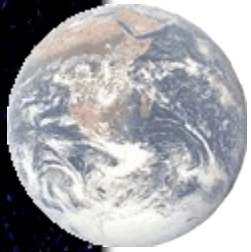
Central themes of O.S. design

- **Agenda:**
 - Designing correct routines for controlling concurrent activities proved to be one of the **most difficult** aspects of systems programming.
 - Difficult to locate errors.
 - Precise conditions under which they appear are very hard to reproduce.



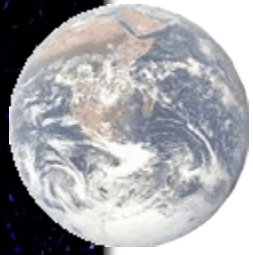
Central Themes of O.S. Design

- Concerned with the management of processes and threads:
- MultiProgramming :
 - multiple processes / A uniprocessor system
- MultiProcessing :
 - multiple processes / multiprocessor
- Distributed processing (Clusters) :
 - multiple processes executing / multiple, distributed computer systems.



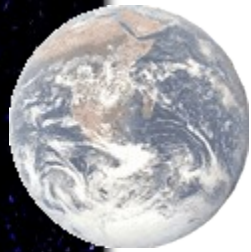
Central Themes of O.S. Design

- Fundamental to OS design: Concurrency.
 - Design issues due to need to achieve **Concurrency**
 - Communication among processes,
 - Sharing of and competing for resources (such as memory, files, and I/O access),
 - synchronization of the activities of multiple processes,
 - Allocation of processor time to processes.



Concurrency arise in diff. contexts

- Multiple applications
 - Multiprogramming
- Structured application
 - Application can be a set of concurrent processes
- Operating-system structure
 - Operating system is a set of processes or threads



Concurrency

Table 5.1 Some Key Terms Related to Concurrency

critical section	A section of code within a process that requires access to shared resources and which may not be executed while another process is in a corresponding section of code.
deadlock	A situation in which two or more processes are unable to proceed because each is waiting for one of the others to do something.
livelock	A situation in which two or more processes continuously change their state in response to changes in the other process(es) without doing any useful work.
mutual exclusion	The requirement that when one process is in a critical section that accesses shared resources, no other process may be in a critical section that accesses any of those shared resources.
race condition	A situation in which multiple threads or processes read and write a shared data item and the final result depends on the relative timing of their execution.
starvation	A situation in which a runnable process is overlooked indefinitely by the scheduler; although it is able to proceed, it is never chosen.



Difficulties of Concurrency

- Same problems arise in multiprogramming / multiprocessing
 - Sharing of global resources : **e.g global variables**
 - Operating system managing the allocation of resources optimally
 - Difficult to locate programming errors
- Reason :
 - relative speed of execution of processes is unpredictable.

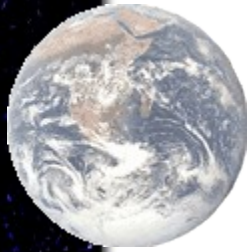


A Simple Example:

consider that we have a **single-processor multiprogramming system** supporting a **single user**

Can be **shared** procedure that is loaded into a portion of memory global to all applications to permit efficient and close interaction among processes.

```
void echo()  
{  
    chin = getchar();  
    chout = chin;  
    putchar(chout);  
}
```



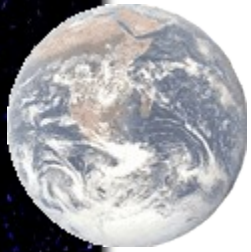
A Simple Example: cont..

The only way to do that is to control the code that accesses the variable.

Impose the **discipline** :

only one process at a time may enter echo and that once in echo the procedure must run to completion before it is available for another process.

Problem: How the discipline can be imposed ?

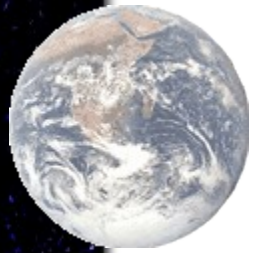


A Simple Example

Process P1

Process P2

```
.      .  
chin = getchar();      .  
      chin = getchar();  
chout = chin;  chout = chin;  
putchar(chout);      .  
.  putchar(chout);  
.  
      .
```



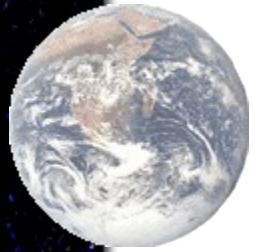
Operating System Concerns

- Keep track of various processes (PCB)
- Allocate and deallocate resources
 - Processor time (Scheduling)
 - Memory (Memory Management)
 - Files (File Management)
 - I/O devices (I/O Management)
- Protect data and resources of each process
- Output of process must be independent of the speed of execution of other concurrent processes



Operating System Concerns

- To understand how the issue of speed independence can be addressed,
- Need to look at the ways in which processes can interact.



Process Interaction

- Processes unaware of each other
- Processes indirectly aware of each other
- Process directly aware of each other



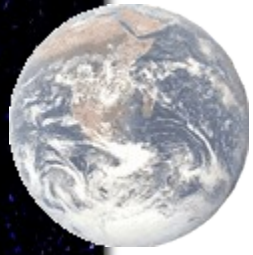
Table 5.2 Process Interaction

Degree of Awareness	Relationship	Influence that one Process has on the Other	Potential Control Problems
Processes unaware of each other	Competition	•Results of one process independent of the action of others •Timing of process may be affected	•Mutual exclusion •Deadlock (renewable resource) •Starvation
Processes indirectly aware of each other (e.g., shared object)	Cooperation by sharing	•Results of one process may depend on information obtained from others •Timing of process may be affected	•Mutual exclusion •Deadlock (renewable resource) •Starvation •Data coherence
Processes directly aware of each other (have communication primitives available to them)	Cooperation by communication	•Results of one process may depend on information obtained from others •Timing of process may be affected	•Deadlock (consumable resource) •Starvation



Competition/Cooperation Among Processes for Resources

- Three control problems must be faced
 - Mutual Exclusion
 - Critical sections
 - Only one program at a time is allowed in its critical section
 - Example only one process at a time is allowed to send command to the printer
- The enforcement of mutual exclusion creates two additional control problems
 - Deadlock (P1 and P2 , R1 and R2)
 - Starvation



Competition Among Processes for Resources

- Control of competition inevitably involves the **OS for allocating resources.**
- In addition, the processes themselves will need to be able to express the requirement for **mutual exclusion** in some fashion,



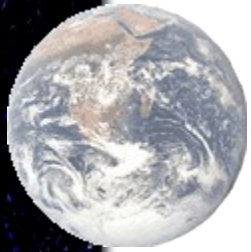
Requirements for Mutual Exclusion

- Any facility or capability that is to provide **support for mutual exclusion** should meet the following requirements:
 - Mutual exclusion must be enforced. i.e
 - Only **one process** at a time is allowed in the **critical section** for a resource.
 - A process that halts in its **noncritical section** must do so without **interfering** with other processes
 - No deadlock or starvation allowed.



Requirements for Mutual Exclusion

- A process must not be delayed access to a critical section when there is no other process using it.
- No assumptions are made about relative process speeds or number of processes.
- A **process** remains inside its **critical section** for a **finite time** only.



Mutual Exclusion (contd..)

- Ways in which the requirements for mutual exclusion can be satisfied :
 - 1. Hardware support.:
 - Interrupt Disabling
 - Special Machine Instruction
 - 2. OS and programming language mechanisms that are used to provide concurrency. (**Semaphores**)
 - 3. To leave the responsibility with the processes that wish to execute concurrently.

Processes would be coordinating with one another to enforce mutual exclusion,

With no support from the programming language or the OS.

-



Mutual Exclusion : Hardware Support

- **Interrupt Disabling (on a uniprocessor system)**
 - A process runs until it invokes an operating system service or until it is interrupted.
 - Disabling interrupts guarantees mutual exclusion.
 - Capability can be provided in the form of **primitives** defined by the OS kernel for **disabling and enabling interrupts**.

```
while (true) {  
    /* disable interrupts */;  
    /* critical section */;  
    /* enable interrupts */;  
    /* remainder */;  
}
```



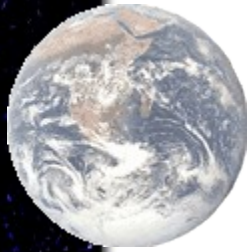
Mutual Exclusion: Hardware Support

- Adv..
 - Simple to implement.
- Disadv..
 - **Expensive** , efficiency of execution could be degraded
 - Processor is limited in its ability to interleave programs.
 - approach will not work in a **multiprocessor architecture**
 - Multiprocessor architecture (**e.g ???**)
 - **disabling interrupts** on one processor will not guarantee mutual exclusion



Mutual Exclusion: Hardware Support

- Special Machine Instructions:
 - Processor designers have proposed several **machine instructions**
 - that carry out two actions **atomically**, such as **reading and writing** or **reading and testing**, of a single memory location with **one instruction fetch cycle**.
 - Access to the memory location is **blocked** for any other instructions **referencing that location**.



Mutual Exclusion: Hardware Support

- **Compare and Swap Instruction**

```
int compare_and_swap ( int *word, int testval,  
                      int newval)
```

```
{
```

```
    int oldval;
```

```
    oldval = *word
```

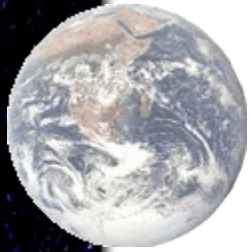
```
    if (oldval == testval) *word = newval;
```

```
    return oldval;
```

```
} This atomic instruction has two parts:
```

- A **compare** is made between a memory value and a test value; if the values are the same, a **swap** occurs.

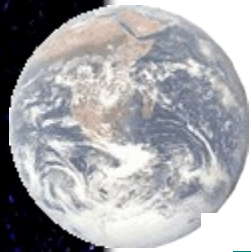
- The entire compare&swap function is carried out atomically—that is, it is not subject to interruption.



Mutual Exclusion: Hardware Support

- **EXCHANGE INSTRUCTION :**
 - The instruction exchanges the contents of a register with that of a memory location.

```
void exchange ( int *register, int *memory)
{
    int temp;
    temp = *memory;
    *memory = *register;
    *register = temp;
}
```



Mutual Exclusion: Hardware Support

```
/* program mutualexclusion */
const int n = /* number of processes */;
int bolt;
void P(int i)
{
    while (true) {
        while (compare_and_swap(bolt, 0, 1) == 1)
            /* do nothing */;
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
}
void main()
{
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}
```

(a) Compare and swap instruction

```
/* program mutualexclusion */
int const n = /* number of processes */;
int bolt;
void P(int i)
{
    int keyi = 1;
    while (true) {
        do exchange (&keyi, &bolt)
        while (keyi != 0);
        /* critical section */;
        bolt = 0;
        /* remainder */;
    }
}
void main()
{
    bolt = 0;
    parbegin (P(1), P(2), ..., P(n));
}
```

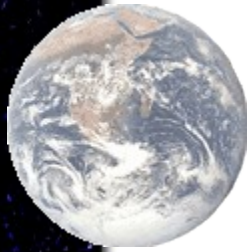
(b) Exchange instruction

Figure 5.2 Hardware Support for Mutual Exclusion



Mutual Exclusion: Hardware Support

- Some version of this instruction is available on nearly all processor families (x86, IA64, sparc, IBM z series, etc.),
- Both the Intel IA-32 architecture (Pentium) and the IA-64 architecture (Itanium) contain an XCHG instruction.
- **Most operating systems use this instruction for support of concurrency.**



Mutual Exclusion: Machine Instructions

- Advantages
 - Applicable to any number of processes on either a single processor or multiple processors sharing main memory.
 - It is simple and therefore easy to verify
 - It can be used to support multiple critical sections



Mutual Exclusion Machine Instructions

- Disadvantages
 - Busy-waiting consumes processor time
 - Starvation is possible when a process leaves a critical section and more than one process is waiting.
 - Deadlock
 - If a low priority process has the critical region and a higher priority process needs, the higher priority process will obtain the processor to wait for the critical region



Mutual Exclusion OS and Prog. Land. based

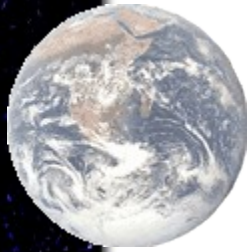
Table 5.3 Common Concurrency Mechanisms

Semaphore	An integer value used for signaling among processes. Only three operations may be performed on a semaphore, all of which are atomic: initialize, decrement, and increment. The decrement operation may result in the blocking of a process, and the increment operation may result in the unblocking of a process. Also known as a counting semaphore or a general semaphore .
Binary Semaphore	A semaphore that takes on only the values 0 and 1.
Mutex	Similar to a binary semaphore. A key difference between the two is that the process that locks the mutex (sets the value to zero) must be the one to unlock it (sets the value to 1).
Condition Variable	A data type that is used to block a process or thread until a particular condition is true.
Monitor	A programming language construct that encapsulates variables, access procedures, and initialization code within an abstract data type. The monitor's variable may only be accessed via its access procedures and only one process may be actively accessing the monitor at any one time. The access procedures are <i>critical sections</i> . A monitor may have a queue of processes that are waiting to access it.
Event Flags	A memory word used as a synchronization mechanism. Application code may associate a different event with each bit in a flag. A thread can wait for either a single event or a combination of events by checking one or multiple bits in the corresponding flag. The thread is blocked until all of the required bits are set (AND) or until at least one of the bits is set (OR).
Mailboxes/Messages	A means for two processes to exchange information and that may be used for synchronization.
Spinlocks	Mutual exclusion mechanism in which a process executes in an infinite loop waiting for the value of a lock variable to indicate availability.



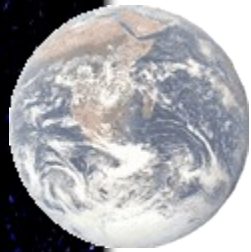
Semaphores

- Two or more processes can **cooperate** by means of simple **signals**,
- In a way that process can be forced to stop at a specified place until it has received a specific signal.
- Special variable called a **semaphore** is used for signaling.
- If a process is waiting for a signal, it is suspended until that signal is sent.



Semaphores

- Semaphore is a variable that has an integer value (**integer semaphore**)
 - May be initialized to a nonnegative number
 - semWait operation decrements the semaphore value (If the value becomes **negative**, then the process executing the semWait is blocked. Otherwise, the process continues execution.)
 - Signal operation increments semaphore value (If the resulting value is **less than or equal to zero**, then a process **blocked** by a semWait operation, if any, is **unblocked**.)



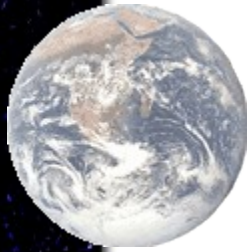
Semaphore Primitives

```
struct semaphore {
    int count;
    queueType queue;
}

void semWait(semaphore s)
{
    s.count--;
    if (s.count < 0)
    {
        place this process in s.queue;
        block this process
    }
}

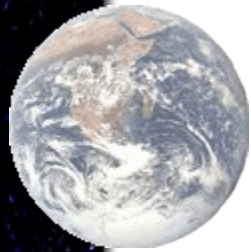
void semSignal(semaphore s)
{
    s.count++;
    if (s.count <= 0)
    {
        remove a process P from s.queue;
        place process P on ready list;
    }
}
```

Figure 5.3 A Definition of Semaphore Primitives



Semaphores

- 1. A binary semaphore may be initialized to 0 or 1.
- 2. The semWaitB operation checks the semaphore value. If the value is **zero**, then the process executing the semWaitB is **blocked**. If the value is **one**, then the value is **changed to zero** and the **process continues execution**.
- 3. The semSignalB operation **checks** to see if any processes are **blocked on this semaphore** (semaphore value equals 0). If so, then a process **blocked** by a semWaitB operation is **unblocked**. If **no processes are blocked**, then the value of the semaphore is **set to one**.



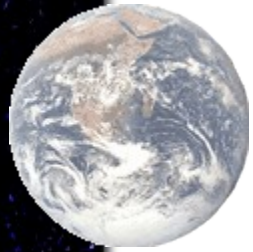
Binary Semaphore Primitives

```
struct binary_semaphore {
    enum {zero, one} value;
    queueType queue;
};

void semWaitB(binary_semaphore s)
{
    if (s.value == 1)
        s.value = 0;
    else
    {
        place this process in s.queue;
        block this process;
    }
}

void semSignalB(semaphore s)
{
    if (s.queue.is_empty())
        s.value = 1;
    else
    {
        remove a process P from s.queue;
        place process P on ready list;
    }
}
```

Figure 5.4 A Definition of Binary Semaphore Primitives



Mutual Exclusion Using Semaphores

```
/* program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1;
void P(int i)
{
    while (true)
    {
        semWait(s);
        /* critical section */
        semSignal(s);
        /* remainder */
    }
}
void main()
{
    parbegin (P(1), P(2), . . . , P(n));
}
```

Figure 5.6 Mutual Exclusion Using Semaphores

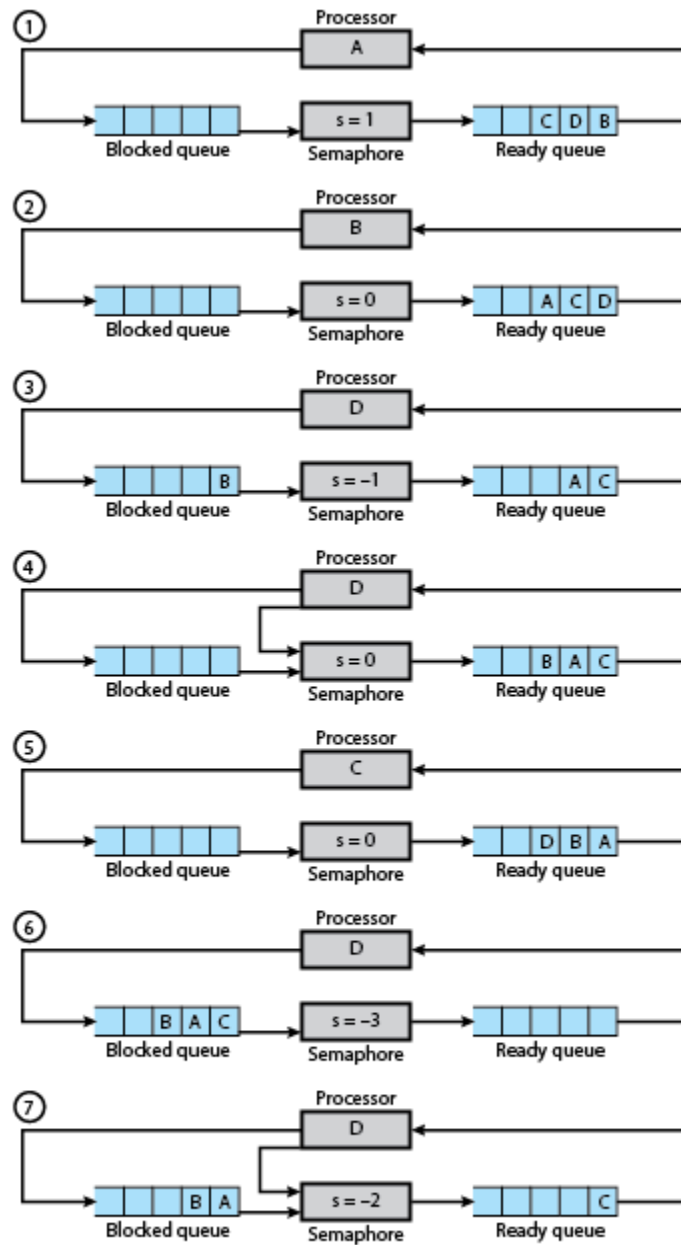
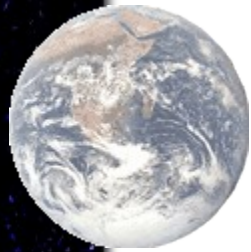


Figure 5.5 Example of Semaphore Mechanism

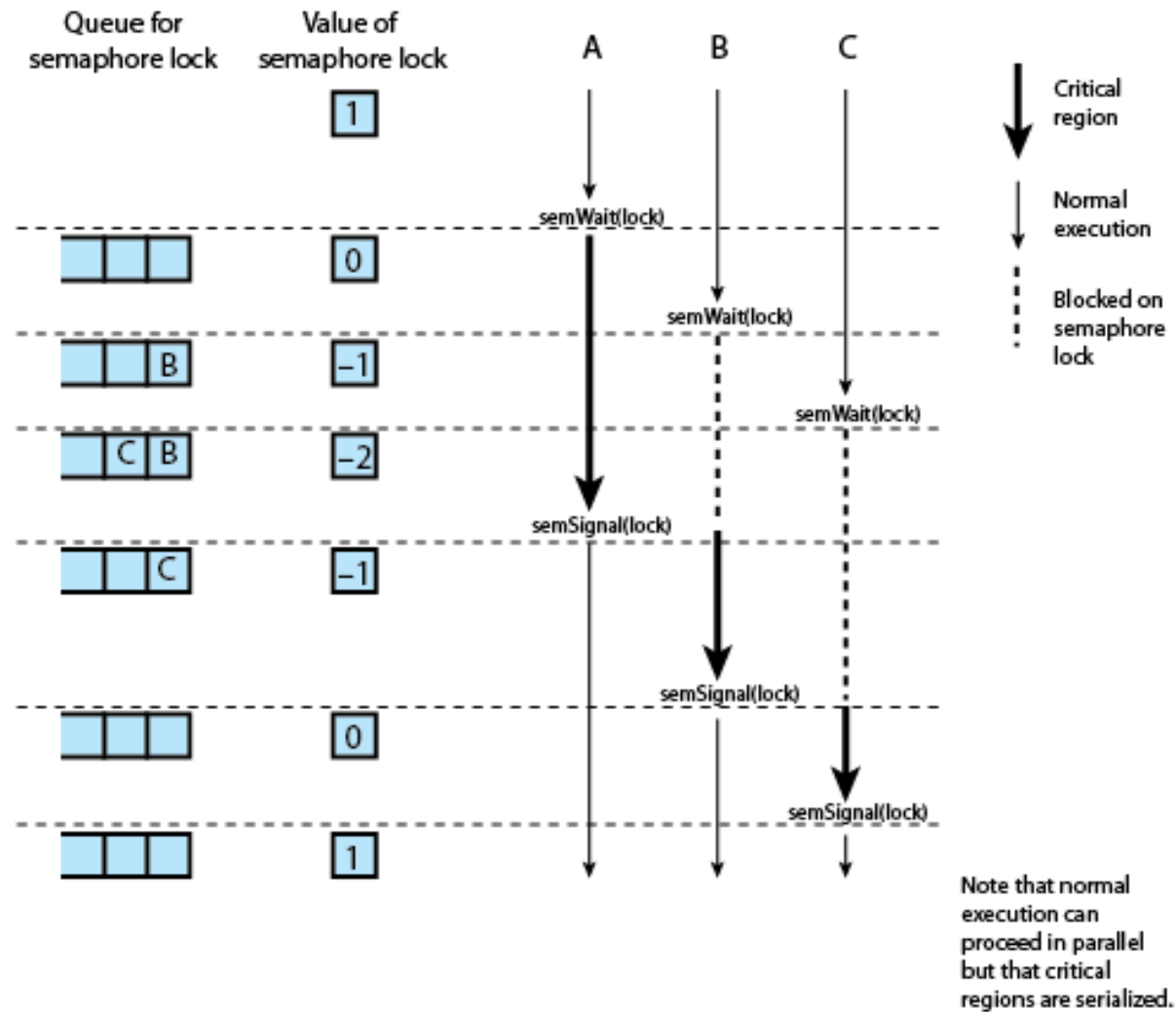
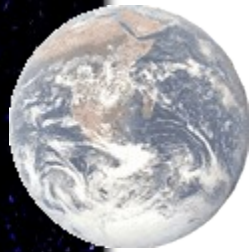
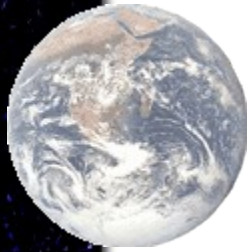


Figure 5.7 Processes Accessing Shared Data Protected by a Semaphore



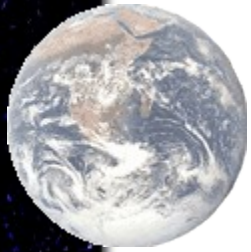
Producer/Consumer Problem

- One or more producers are generating data and placing these in a buffer
- A single consumer is taking items out of the buffer one at time
- Only one producer or consumer may access the buffer at any one time



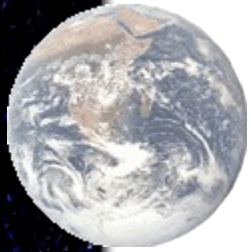
Producer

```
producer:  
while (true) {  
    /* produce item v */  
    b[in] = v;  
    in++;  
}
```

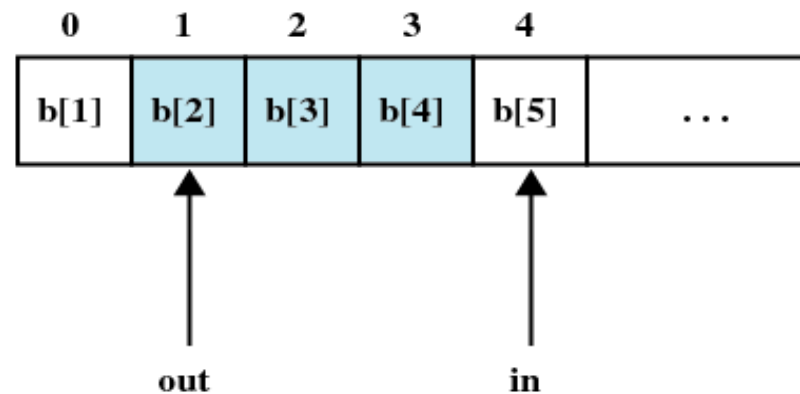


Consumer

```
consumer:
while (true) {
    while (in <= out)
        /*do nothing */;
    w = b[out];
    out++;
    /* consume item w */
}
```

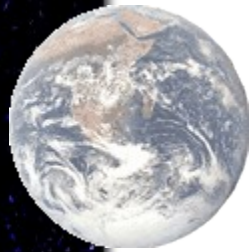


Producer/Consumer Problem



Note: shaded area indicates portion of buffer that is occupied

Figure 5.8 Infinite Buffer for the Producer/Consumer Problem



```
/* program producerconsumer */
int n;
binary_semaphore s = 1;
binary_semaphore delay = 0;
void producer()
{
    while (true)
    {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n==1)
            semSignalB(delay);
        semSignalB(s);
    }
}
void consumer()
{
    semWaitB(delay);
    while (true)
    {
        semWaitB(s);
        take();
        n--;
        semSignalB(s);
        consume();
        if (n==0)
            semWaitB(delay);
    }
}
void main()
{
    n = 0;
    parbegin (producer, consumer);
}
```

Figure 5.9 An Incorrect Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores

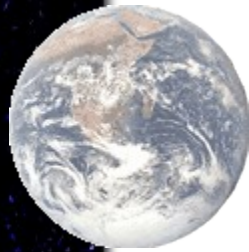


Table 5.4 Possible Scenario for the Program of Figure 5.9

	Producer	Consumer	s	n	Delay
1			1	0	0
2	semWaitB(s)		0	0	0
3	n++		0	1	0
4	if (n==1) (semSignalB(delay))		0	1	1
5	semSignalB(s)		1	1	1
6		semWaitB(delay)	1	1	0
7		semWaitB(s)	0	1	0
8		n--	0	0	0
9		semSignalB(s)	1	0	0
10	semWaitB(s)		0	0	0
11	n++		0	1	0
12	if (n==1) (semSignalB(delay))		0	1	1
13	semSignalB(s)		1	1	1
14		if (n==0) (semWaitB(delay))	1	1	1
15		semWaitB(s)	0	1	1
16		n--	0	0	1
17		semSignalB(s)	1	0	1
18		if (n==0) (semWaitB(delay))	1	0	0
19		semWaitB(s)	0	0	0
20		n--	0	-1	0
21		semSignalB(s)	1	-1	0

Note: White areas represent the critical section controlled by semaphore s.

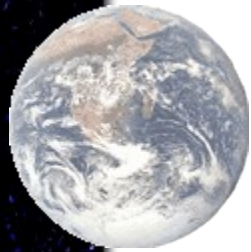


- **Problem:**

- consumer is consuming the item that does not exist.

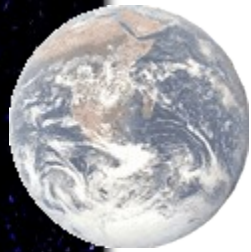
- **Solution:**

- Moving the test in CS ?????
 - (yes??? -- deadlock after stmt 8)
 - No ???
- Taking a local variable
- Using general semaphore



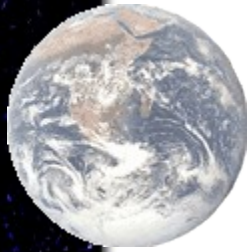
```
/* program producerconsumer */
int n;
binary_semaphore s = 1;
binary_semaphore delay = 0;
void producer()
{
    while (true)
    {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n==1) semSignalB(delay);
        semSignalB(s);
    }
}
void consumer()
{
    int m; /* a local variable */
    semWaitB(delay);
    while (true)
    {
        semWaitB(s);
        take();
        n--;
        m = n;
        semSignalB(s);
        consume();
        if (m==0) semWaitB(delay);
    }
}
void main()
{
    n = 0;
    parbegin (producer, consumer);
}
```

Figure 5.10 A Correct Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores



```
/* program producerconsumer */
semaphore n = 0;
semaphore s = 1;
void producer()
{
    while (true)
    {
        produce();
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
void consumer()
{
    while (true)
    {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        consume();
    }
}
void main()
{
    parbegin (producer, consumer);
}
```

Figure 5.11 A Solution to the Infinite-Buffer Producer/Consumer Problem Using Semaphores



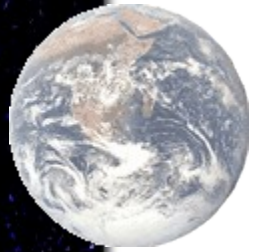
- Whatif ???
 - semSignal(s) and semSignal(n) operations are accidentally reversed in producer.
 - This would require that the **semSignal(n)** operation be performed in the producer's critical section without interruption by the **consumer or another producer.**
- Whatif ???
 - semWait(n) and semWait(s) operations are accidentally reversed
 - Produces a fatal flaw.
- Reason:--
 - If the consumer ever enters its critical section when the buffer is empty ($n.count = 0$), then
 - no producer can ever append to the buffer and the system is deadlocked.
- Major Disadv of Semaphores:
 - Difficulty of producing correct designs.
 - semWait() and semSignal() operations may be scattered throughout a program and it is not easy to see the overall effect of these operations on the semaphores they affect.



Producer with Circular Buffer

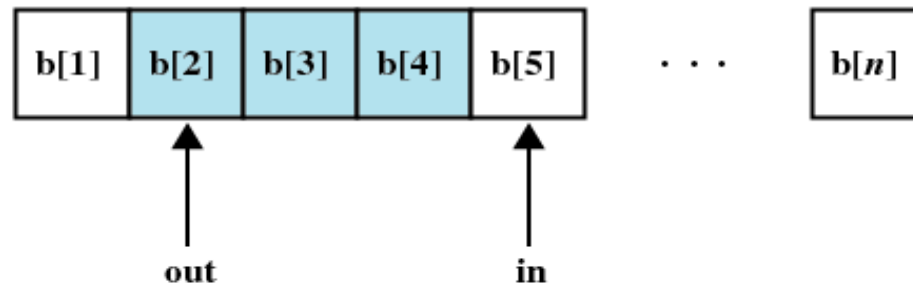
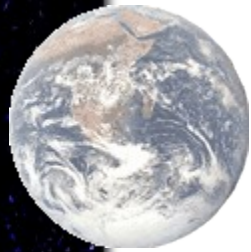
producer:

```
while (true) {  
    /* produce item v */  
    while ((in + 1) % n == out)  
        /* do nothing */;  
    b[in] = v;  
    in = (in + 1) % n  
}
```

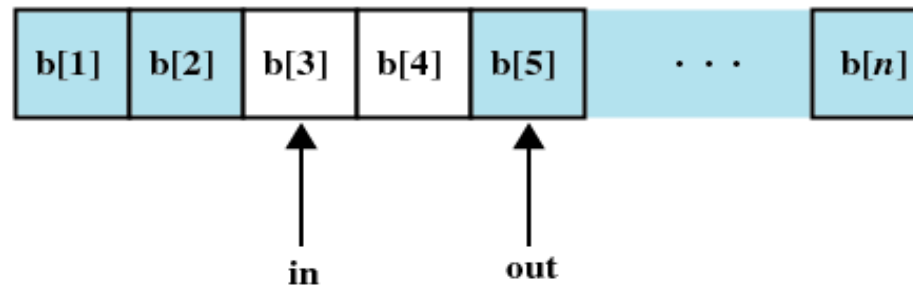


Consumer with Circular Buffer

```
consumer:
while (true) {
    while (in == out)
        /* do nothing */;
    w = b[out];
    out = (out + 1) % n;
    /* consume item w */
}
```

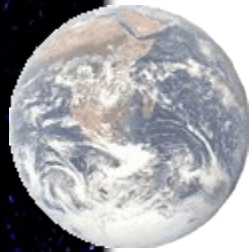


(a)



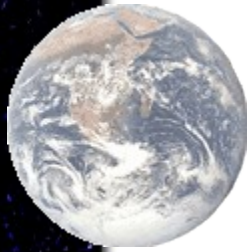
(b)

Figure 5.12 Finite Circular Buffer for the Producer/Consumer Problem



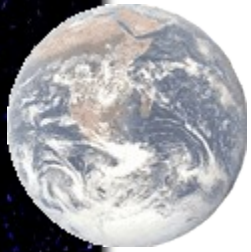
```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1;
semaphore n= 0;
semaphore e= sizeofbuffer;
void producer()
{
    while (true)
    {
        produce();
        semWait(e);
        semWait(s);
        append();
        semSignal(s);
        semSignal(n)
    }
}
void consumer()
{
    while (true)
    {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        semSignal(e);
        consume();
    }
}
void main()
{
    parbegin (producer, consumer);
}
```

Figure 5.13 A Solution to the Bounded-Buffer Producer/Consumer Problem Using Semaphores



Monitors

- Monitor is a programming language construct, software module consisting of :
 - one or more procedures,
 - an initialization sequence, and local data.
- Chief characteristics :
 - Local data variables are accessible only by the monitor's procedure and not by external procedures
 - Process enters monitor by invoking one of its procedures.
 - Only one process may be executing in the monitor at a time.
 - Others who invoke the monitor are blocked, waiting for the monitor to become available.



- By enforcing the discipline of one process at a time, the monitor is able to provide a **mutual exclusion facility**.
- But.... For Concurrent processing
 - Support for synchronization is needed (**Reason???**)
- use of **condition variables** that are:
 - contained within the monitor and accessible only within the monitor.
- **Condition variables** : A special data type in monitors, which are operated on by two functions:
 - cwait(c) : Suspend execution of the calling process on condition **c**
The monitor is now available for use by another process.
 - csignal(c) : Resume execution of some process blocked after a cwait on the same condition **c**. If there are several such processes, choose one of them; if there is no such process, do nothing.

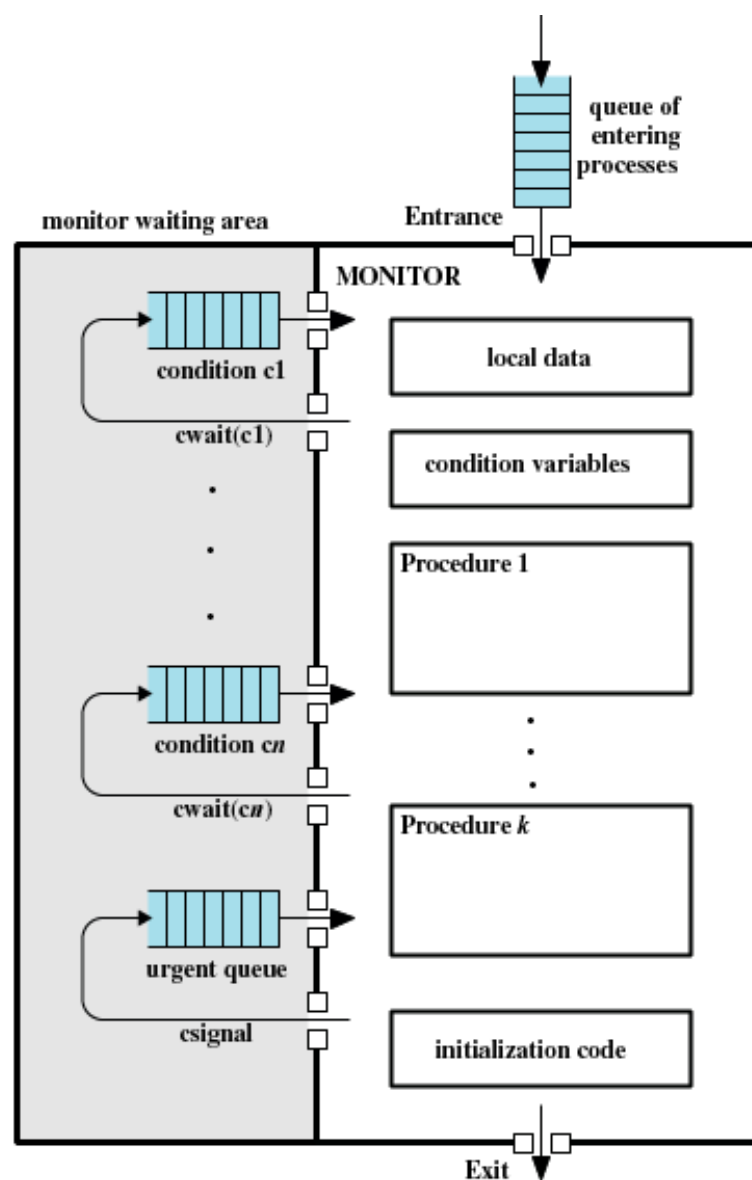
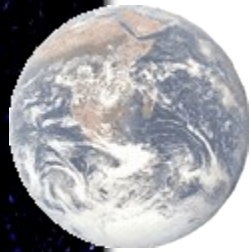
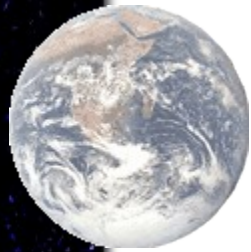


Figure 5.15 Structure of a Monitor

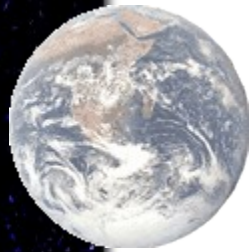


```
/* program producerconsumer */
monitor boundedbuffer;
char buffer [N];                                /* space for N items */
int nextin, nextout;                             /* buffer pointers */
int count;                                       /* number of items in buffer */
cond notfull, notempty;                        /* condition variables for synchronization */

void append (char x)
{
    if (count == N)                             /* buffer is full; avoid overflow */
        cwait(notfull);
    buffer[nextin] = x;
    nextin = (nextin + 1) % N;
    count++;
    /* one more item in buffer */
    csignal(notempty);                          /* resume any waiting consumer */
}

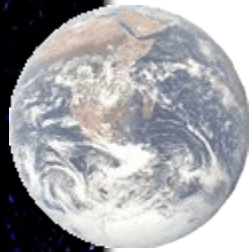
void take (char x)
{
    if (count == 0)                             /* buffer is empty; avoid underflow */
        cwait(notempty);
    x = buffer[nextout];
    nextout = (nextout + 1) % N;
    count--;
    /* one fewer item in buffer */
    csignal(notfull);                          /* resume any waiting producer */
}

/* monitor body */
{
    nextin = 0; nextout = 0; count = 0;          /* buffer initially empty */
}
```



```
void producer()
char x;
{
    while (true)
    {
        produce(x);
        append(x);
    }
}
void consumer()
{
    char x;
    while (true)
    {
        take(x);
        consume(x);
    }
}
void main()
{
    parbegin (producer, consumer);
}
```

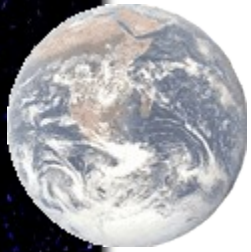
Figure 5.16 A Solution to the Bounded-Buffer Producer/Consumer Problem Using a Monitor



```
void append (char x)
{
    while(count == N)
        cwait(notfull);           /* buffer is full; avoid overflow */
    buffer[nextin] = x;
    nextin = (nextin + 1) % N;
    count++;                       /* one more item in buffer */
    cnotify(notempty);           /* notify any waiting consumer */
}

void take (char x)
{
    while(count == 0)
        cwait(notempty);         /* buffer is empty; avoid underflow */
    x = buffer[nextout];
    nextout = (nextout + 1) % N;
    count--;                       /* one fewer item in buffer */
    cnotify(notfull);           /* notify any waiting producer */
}
```

Figure 5.17 Bounded Buffer Monitor Code for Mesa Monitor

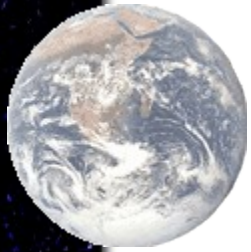


Message Passing

- Two fundamental requirements to be satisfied by system when process interact:
 - synchronization
 - communication.
- Enforce mutual exclusion
- Exchange information

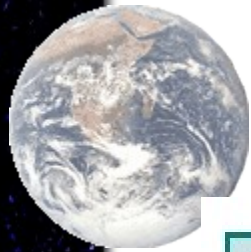
send (destination, message)

receive (source, message)



Message Passing

- Adv:
- It lends itself to implementation in all category of systems
 - distributed systems
 - Shared-memory multiprocessor and uniprocessor systems.
- **Design Characteristics of Message Systems for Inter-process Communication and Synchronization?????**



Message Passing

Synchronization

Send

blocking

nonblocking

Receive

blocking

nonblocking

test for arrival

Addressing

Direct

send

receive

explicit

implicit

Indirect

static

dynamic

ownership

Format

Content

Length

fixed

variable

Queueing Discipline

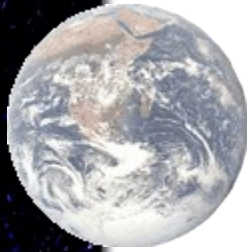
FIFO

Priority



Synchronization

- Sender and receiver may or may not be blocking (waiting for message)
- Blocking send, blocking receive
 - Both sender and receiver are blocked until message is delivered
 - Called a rendezvous



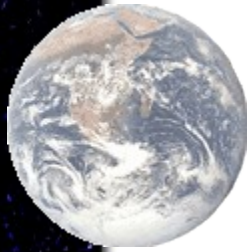
Synchronization

- Nonblocking send, blocking receive
 - Sender continues on
 - Receiver is blocked until the requested message arrives
- Nonblocking send, nonblocking receive
 - Neither party is required to wait



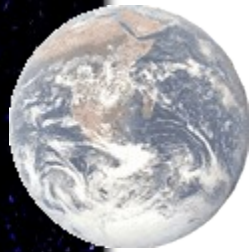
Addressing

- Direct addressing
 - Send primitive includes a specific identifier of the destination process
 - Receive primitive could know ahead of time which process a message is expected
 - Receive primitive could use source parameter to return a value when the receive operation has been performed

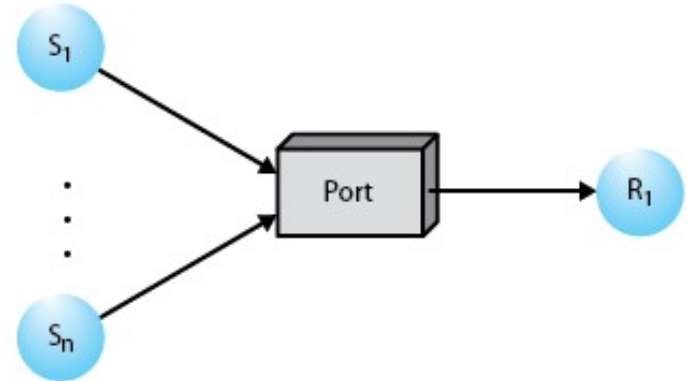


Addressing

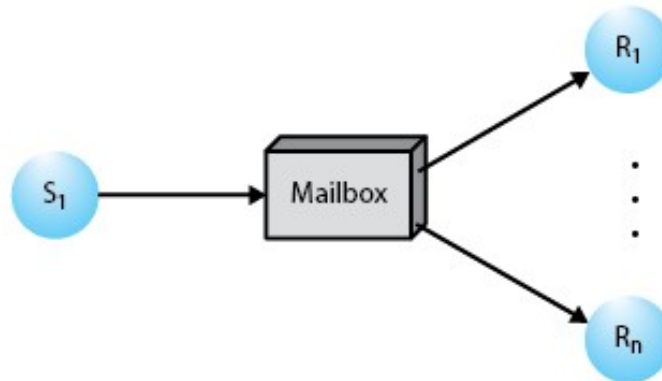
- Indirect addressing
 - Messages are sent to a shared data structure consisting of queues
 - Queues are called mailboxes
 - One process sends a message to the mailbox and the other process picks up the message from the mailbox



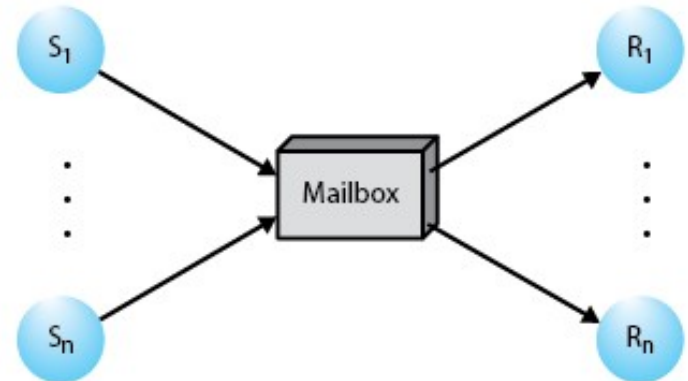
(a) One to one



(b) Many to one

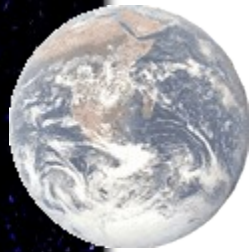


(c) One to many



(d) Many to many

Figure 5.18 Indirect Process Communication



Message Format

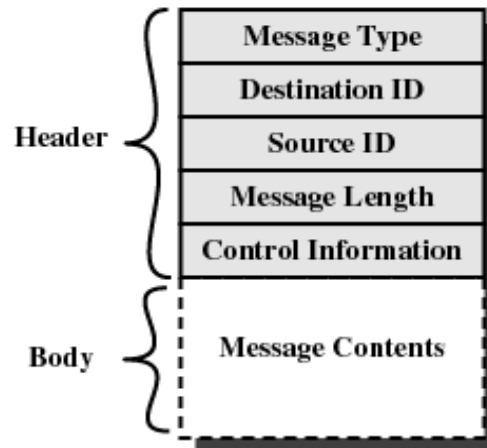
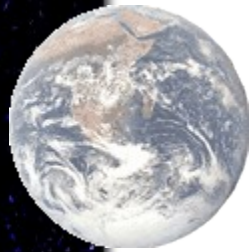
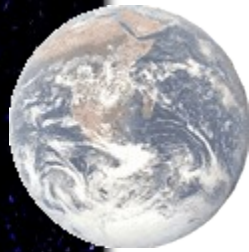


Figure 5.19 General Message Format



```
/* program mutualexclusion */
const int n = /* number of processes */;
void P(int i)
{
    message msg;
    while (true)
    {
        receive (mutex, msg);
        /* critical section */;
        send (mutex, msg);
        /* remainder */;
    }
}
void main()
{
    create_mailbox (mutex);
    send (mutex, null);
    parbegin (P(1), P(2), . . . , P(n));
}
```

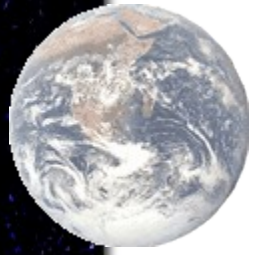
Figure 5.20 Mutual Exclusion Using Messages



```
const int
    capacity = /* buffering capacity */ ;
    null = /* empty message */ ;
int i;
void producer()
{   message pmsg;
    while (true)
    {
        receive (mayproduce, pmsg);
        pmsg = produce();
        send (mayconsume, pmsg);
    }
}
void consumer()
{   message cmsg;
    while (true)
    {
        receive (mayconsume, cmsg);
        consume (cmsg);
        send (mayproduce, null);
    }
}

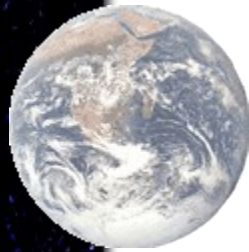
void main()
{
    create_mailbox (mayproduce);
    create_mailbox (mayconsume);
    for (int i = 1; i <= capacity; i++)
        send (mayproduce, null);
    parbegin (producer, consumer);
}
```

Figure 5.21 A Solution to the Bounded-Buffer Producer/Consumer Problem Using Messages



Readers/Writers Problem

- Any number of readers may simultaneously read the file
- Only one writer at a time may write to the file
- If a writer is writing to the file, no reader may read it



```
/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
    while (true)
    {
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0)
            semSignal (wsem);
        semSignal (x);
    }
}
void writer()
{
    while (true)
    {
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
    }
}
void main()
{
    readcount = 0;
    parbegin (reader, writer);
}
```

Figure 5.22 A Solution to the Readers/Writers Problem Using Semaphores: Readers Have Priority



```
/*program readersandwriters*/
int readcount, writecount;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;
void reader()
{
    while (true)
    {
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0)
            semSignal (wsem);
        semSignal (x);
    }
}
void writer ()
{
    while (true)
    {
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0)
            semSignal (rsem);
        semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}
```

Figure 5. 23 A Solution to the Readers/Writers Problem Using Semaphores: Writers Have Priority